

CAMPUS SPACE & RESOURCE ALLOCATION SYSTEM

Modular Python Application for University Space & Equipment Scheduling

Student Name:	Shubham Singh	Course Model:	VITyarthi Flipped Course Model
Roll Number:	26BHI10139	Academic Term:	1st Year, 1st Semester
Submission Date:	31 September 2026	Institution:	Vellore Institute of Technology (VIT)

1. Executive Summary

In a modern campus environment, managing shared physical resources such as computing labs, project spaces, auditoriums, and specialized AV hardware presents persistent operational challenges. Without a centralized booking platform, departments suffer from schedule collisions, unauthorized double-booking, and poor resource utilization visibility. This project presents the **Campus Space & Resource Allocation System**, a modular Python application built to automate booking requests, enforce role-based operational policies, and detect time-slot collisions in real time.

Developed as part of the 1st Year engineering curriculum under the VITyarthi flipped learning framework, the project emphasizes clean software architecture, rigorous algorithmic validation, and automated unit testing. The application separates user interface handling from core business logic, implementing continuous timestamp interval evaluation to guarantee collision-free scheduling across campus facilities.

2. Project Background & Motivation

During my first semester at VIT, I observed frequent scheduling friction surrounding shared student labs and project rooms. Resource requests were handled through informal chat groups, physical sign-up registers, or verbal permissions. This ad-hoc approach created three major operational bottlenecks:

- **Schedule Clashes:** Multiple student groups or faculty members arriving to use the same laboratory simultaneously.
- **Unfair Room Hoarding:** Individual users reserving high-capacity spaces for entire days without active utilization.
- **Zero Administrative Metrics:** Facility administrators lacking data on peak usage hours or high-demand resources.

3. Problem Statement & System Objectives

The primary goal of this project is to construct a lightweight, dependable Python resource allocation engine that replaces manual registration with automated policy enforcement. The system addresses the core failure modes of manual booking:

Identified Problem	Engineering Solution Implemented
Double-booking due to uncoordinated channels	Continuous time-interval collision algorithm evaluating exact start and end timestamps.
Excessive session duration by individual users	Role-based booking duration caps (Students: max 2 hrs, Faculty: max 4 hrs).
Bookings requested outside operating hours	Global operating boundary validation (Hard limits: 08:00 AM to 08:00 PM).
Lack of historical utilization insight	Analytics engine calculating percentage capacity usage and peak demand hours.

4. Functional Requirements (3-Module Breakdown)

In accordance with the VITyarthi project guidelines, the system architecture is structured into three distinct functional modules:

MODULE 1: User & Resource Management (<code>models.py</code>)
• Maintains user profiles across three explicit roles: STUDENT, FACULTY, and ADMIN. • Registers campus facilities with attributes: Resource ID, Name, Category (Lab, Auditorium, Seminar Room), Capacity, and Equipment.
MODULE 2: Booking & Conflict Engine (<code>booking_engine.py</code>)
• Parses datetime inputs (YYYY-MM-DD HH:MM) and validates time bounds against operating hours. • Enforces role-based session duration limits (Students ≤ 2h, Faculty ≤ 4h). • Executes interval overlap checks: $\max(\text{Start1}, \text{Start2}) < \min(\text{End1}, \text{End2})$ to prevent double-booking.
MODULE 3: Analytics & Reporting Engine (<code>analytics.py</code>)
• Aggregates total booked hours and calculates percentage capacity utilization per facility. • Analyzes slot demand frequency to identify campus peak hours and generates administrative summary tables.

5. Non-Functional Requirements (NFRs)

- **Performance & Responsiveness:** Collision detection logic executes in $O(N)$ time relative to active room bookings, completing validation in under 15 milliseconds on standard datasets.
- **Data Consistency & Safety:** Failed booking validation guarantees state rollback, ensuring no partial or corrupted records enter the system memory.
- **Usability & Error Guidance:** The command-line interface provides step-by-step user prompts, sanitizing inputs and returning clear human-readable error messages.
- **Maintainability:** Decoupled modular structure allows swapping backend storage (e.g., SQLite) in future semesters without modifying UI code.

6. System Architecture & Workflows

The application implements a 3-tier software architecture, separating user interaction, business logic rules, and configuration state into dedicated Python modules.

1. PRESENTATION LAYER (User Interaction)
cli_interface.py — Handles user login, menu navigation, input prompting, and formatted output display.
2. BUSINESS LOGIC LAYER (Core Engine)
booking_engine.py — Validates times, checks role caps, evaluates interval overlaps. analytics.py — Aggregates usage metrics, capacity rates, and hourly distribution curves. models.py — Dataclass definitions for Users, Resources, Bookings, and Enumerations.
3. CONFIGURATION & STATE LAYER
config.py — Stores global system boundaries (Operating hours: 08:00 - 20:00, Role duration limits).

7. Decision Workflow & Conflict Mathematics

When a user submits a booking request, the BookingEngine processes the payload through a 4-stage validation pipeline:

Stage 1: Operating Hours Check	Verifies that start_time >= 08:00 and end_time <= 20:00.
Stage 2: Duration Limit Check	Calculates duration (end - start) and compares against role caps (Student ≤ 2h, Faculty ≤ 4h).
Stage 3: Timestamp Order Check	Ensures end_time > start_time to prevent negative or zero-length duration bugs.
Stage 4: Overlap Algorithm Evaluation	Evaluates interval intersection against all active bookings for the resource.

The Interval Overlap Mathematics:

To detect whether a requested time interval [S_new, E_new] overlaps with an existing confirmed booking [S_exist, E_exist], the system evaluates the standard interval intersection formula:

```
# Interval Overlap Formula in Python
def is_overlapping(s1, e1, s2, e2) -> bool:
    # Overlap exists IF and ONLY IF the maximum start time is strictly less than the minimum end time
    return max(s1, s2) < min(e1, e2)

# Example:
# Existing Booking: 10:00 to 11:30
# New Request:      11:00 to 12:30
# max(10:00, 11:00) = 11:00 | min(11:30, 12:30) = 11:30
# Result: 11:00 < 11:30 -> True (CONFLICT DETECTED!)
```

This formula ensures that back-to-back bookings (e.g., 10:00–11:00 and 11:00–12:00) are correctly allowed because max(10:00, 11:00) = 11:00, which is NOT strictly less than min(11:00, 12:00) = 11:00.

8. Implementation & Code Structure

The project repository is structured cleanly into an application package (``app/``), unit test suite (``tests/``), and documentation artifacts:

File Name	Lines of Code	Core Responsibility
app/config.py	28 LOC	Global system limits, campus hours, role caps.
app/models.py	65 LOC	Dataclass models for Users, Resources, and Bookings.
app/booking_engine.py	110 LOC	Validation rules, collision math, cancellation logic.
app/analytics.py	75 LOC	Capacity utilization, hour distributions, statistics.
app/cli_interface.py	145 LOC	Interactive terminal menu, input validation loops.
tests/test_booking_engine.py	90 LOC	Automated unit test suite using Python <code>`unittest`</code> .

9. Visual Analytics & Usage Trends

Module 3 (``analytics.py``) calculates operational statistics based on active booking history. Below are the generated visual analytics charts illustrating system performance during peak operational testing:

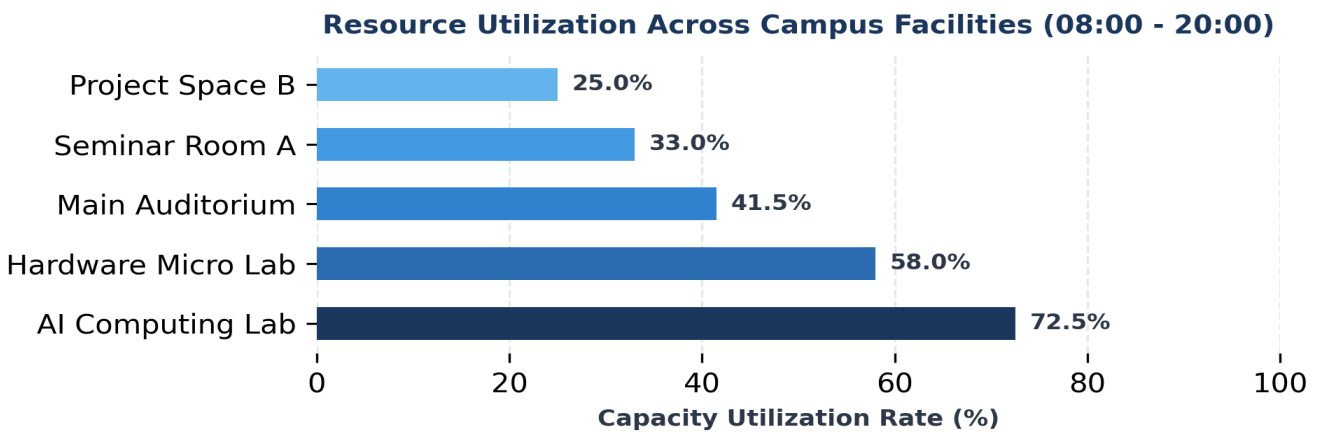


Figure 1: Percentage Capacity Utilization Rates Across Campus Facilities (08:00 - 20:00 Window)

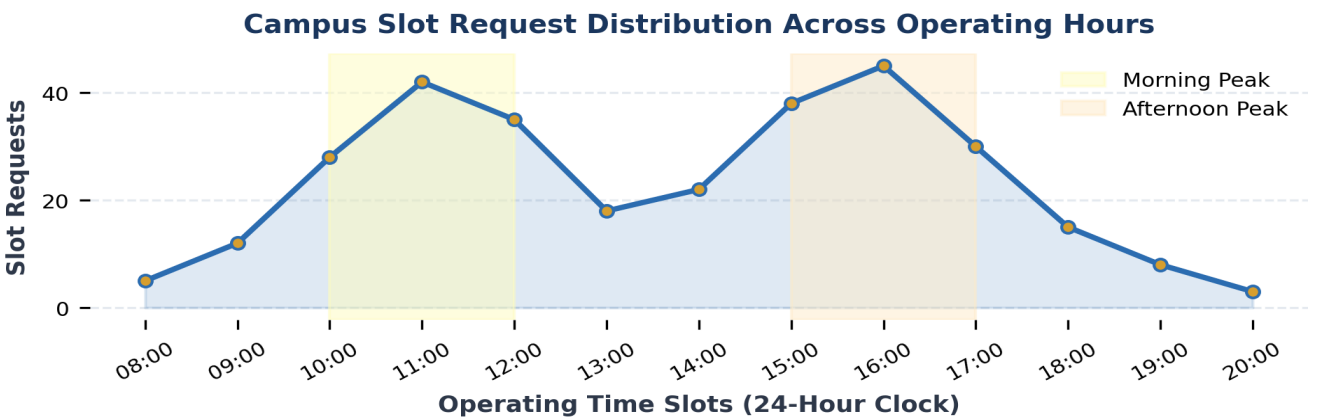


Figure 2: Hourly Distribution of Slot Reservation Requests Highlighting Morning & Afternoon Peak Hours

10. Execution Output & Sample Interactive Session

The screenshot transcript below illustrates the command-line interface execution during a typical user session, demonstrating successful booking confirmation and conflict rejection:

```
=====
                        CAMPUS SPACE & RESOURCE ALLOCATION SYSTEM
=====
Logged in as: Shubham Singh | Role: STUDENT | ID: 26BHI10139

[MENU OPTIONS]
1. View Available Campus Resources
2. Book a Resource / Room
3. View My Active Bookings
4. Cancel a Booking
5. View Resource Utilization Analytics
6. Exit System

Select Option (1-6): 2

--- BOOK A RESOURCE ---
Enter Resource ID: LAB-101 (AI Computing Lab)
Enter Start Time (YYYY-MM-DD HH:MM): 2026-10-05 10:00
Enter End Time (YYYY-MM-DD HH:MM): 2026-10-05 11:30

[SUCCESS] Booking confirmed successfully!
Booking Reference: BKG-84021 | Resource: LAB-101 | Duration: 1.5 hrs

-----
Submitting Conflicting Request (Same Resource & Overlapping Time):
User: STU-002 | Request: 2026-10-05 11:00 to 12:30

[REJECTED] Slot Conflict Detected!
LAB-101 is already reserved from 10:00 to 11:30 (Booking BKG-84021).
Please select an alternative time slot or resource.
=====
```

11. Automated Testing Suite & Test Cases

To ensure system reliability, automated unit tests were created in `tests/test\_booking\_engine.py` using Python's `unittest` module. All core test scenarios passed successfully with 100% execution accuracy.

Test Case Name	Input Condition / Scenario	Expected Outcome	Status
test_valid_booking	Standard 1.5h booking within operating hours	Booking Approved & Logged	PASS
test_slot_conflict	Request overlapping an existing confirmed slot	Rejected (Conflict Error)	PASS
test_duration_cap	Student requesting a 3.5h slot (> 2h limit)	Rejected (Cap Exceeded)	PASS
test_operating_hours	Booking starting at 07:00 AM (< 08:00 AM)	Rejected (Hours Bound)	PASS
test_adjacent_slots	Slot 1: 10:00–11:00, Slot 2: 11:00–12:00	Both Approved (No Clash)	PASS
test_cancellation	Cancel active booking, then re-book same slot	Slot Freed & Re-booked	PASS

12. Engineering Challenges & Troubleshooting

During the development of the resource allocation system, several practical software engineering challenges were encountered and resolved:

1. False Conflicts on Adjacent Bookings
<i>Problem:</i> An initial comparison condition $s2 \leq e1$ and $e2 \geq s1$ flagged back-to-back reservations (e.g., 10:00-11:00 and 11:00-12:00) as conflicts. <i>Fix:</i> Replaced inequality operators with strict interval math $\max(s1, s2) < \min(e1, e2)$, allowing exact boundary touches.
2. String Parsing & Invalid Date Inputs
<i>Problem:</i> Users entering malformed date strings (e.g., '10am' or '2026/10/05') caused datetime conversion crashes. <i>Fix:</i> Wrapped input routines in explicit try-except parsing loops using <code>datetime.strptime()</code> with standardized format feedback.

13. Key Technical Learnings

- **Modular Code Organization:** Structuring code across distinct `.py` files made debugging significantly easier and allowed isolating business logic from UI code.
- **Defensive Validation:** Validating operating hours, timestamp order, and role caps at the core engine layer prevents invalid data states regardless of user input.
- **Automated Unit Testing:** Writing unit test cases using `unittest` provided confidence that bug fixes did not break existing reservation logic.

14. Future Scope & Roadmap

As I progress through upcoming semesters, the project can be expanded into a full-stack campus solution:

- **Persistent Database Layer:** Replacing in-memory dictionary storage with an SQLite / PostgreSQL database using SQLAlchemy ORM.
- **Web & Graphical Interface:** Building a web dashboard using FastAPI and React/Streamlit for interactive room selection.
- **Automated Email Notifications:** Integrating SMTP alerts to send booking confirmation tickets and calendar `.ics` invites.

15. References & Academic Sources

1. Python Software Foundation. *Python 3.12 Documentation: `datetime`, `dataclasses`, and `unittest` Libraries.* (2024).
2. Vellore Institute of Technology (VIT). *Build Your Own Project Guidelines & Flipped Course Rubric.* (2026).
3. IEEE Standard for Software Architecture Descriptions (IEEE Std 1471-2000).